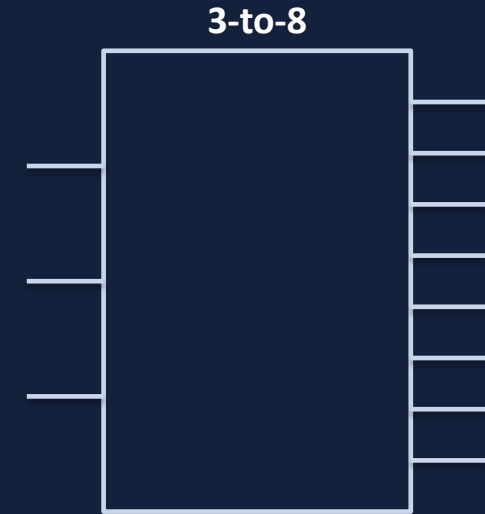


Lecture 12

Decoders and Encoders

Decoders and enables · active-low outputs · decoder expansion
· functions from a decoder · encoders and priority encoders

Reading: Mano & Ciletti, *Digital Design with an Introduction to the Verilog HDL*, 6th ed. — Chapter 4, §4.9–4.10



Today's roadmap

1

§4.9

Decoders

One output per minterm

2

§4.9

Expansion

Enables build bigger decoders

3

§4.9

Functions

Decoder + OR = any function

4

§4.10

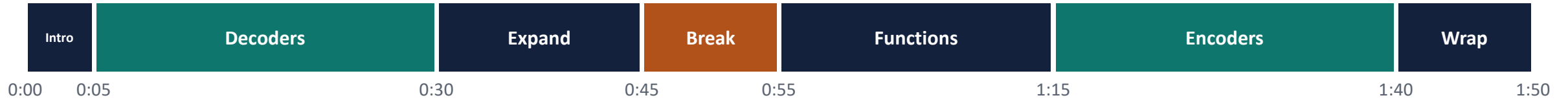
Encoders

Priority when inputs collide

By the end of class you can...

- Build an n -to- 2^n decoder with an enable from gates
- Read active-low decoder tables with don't-care entries
- Combine decoders into a larger decoder
- Implement several functions with one decoder
- Design a priority encoder with a valid output

Agenda for today's session

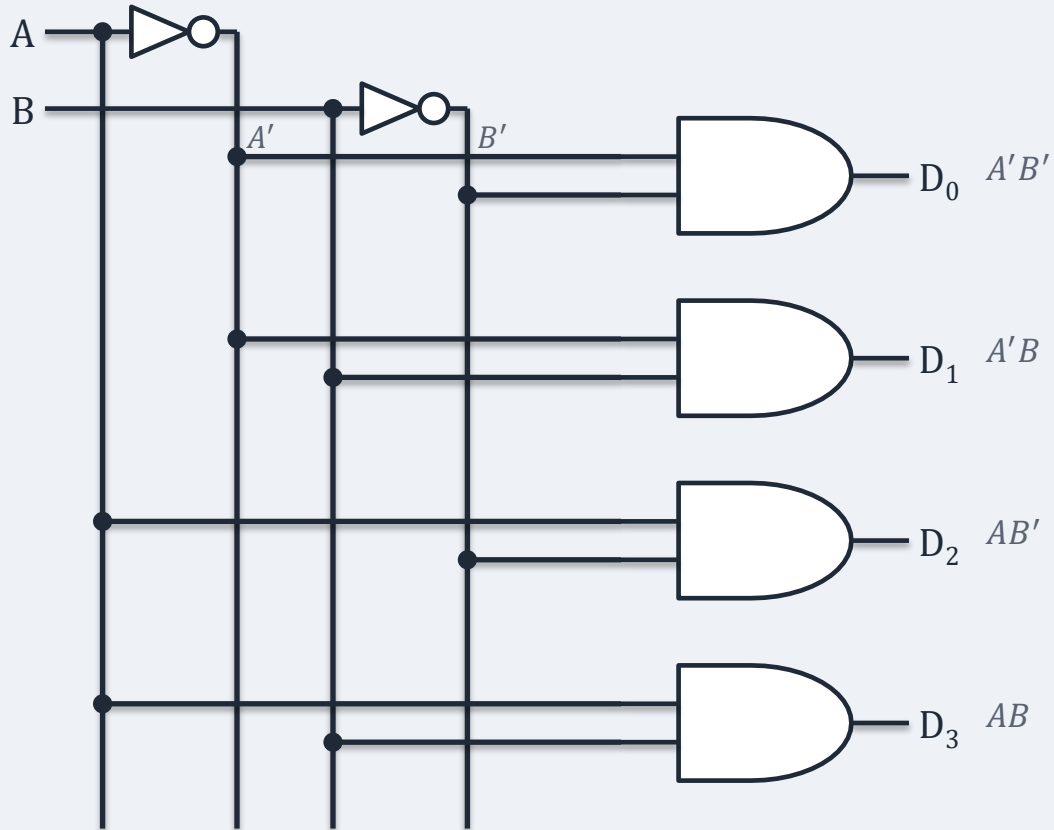


Elapsed time	Topic	In-class work
0:00 – 0:05	Welcome, roadmap	—
0:05 – 0:30	Gate-level decoders, enables, active-low	Example 1
0:30 – 0:45	Decoder expansion	Example 2
0:45 – 0:55	Break	
0:55 – 1:15	Functions with a decoder	Example 3 · Your turn 1
1:15 – 1:40	Encoders and priority encoders	Example 4 · Your turn 2
1:40 – 1:50	Verilog decoder; wrap-up	Lab F preview

The 2-to-4 line decoder

§4.9

Gate level, active-high outputs



A	B	D ₀	D ₁	D ₂	D ₃
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

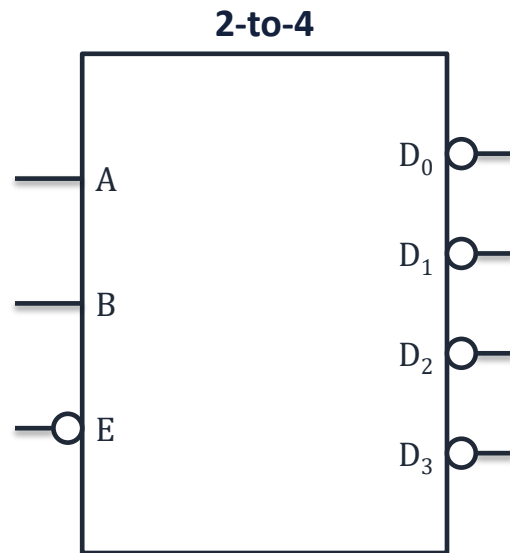
Each output is one minterm

An n -to- 2^n decoder generates all 2^n minterms of its inputs.

Exactly one output is 1 for each input code.

Enable and active-low outputs

§4.9



Bubbles mark active-low pins: an output asserted as 0, an enable that works when 0.

E'	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

Reading the table

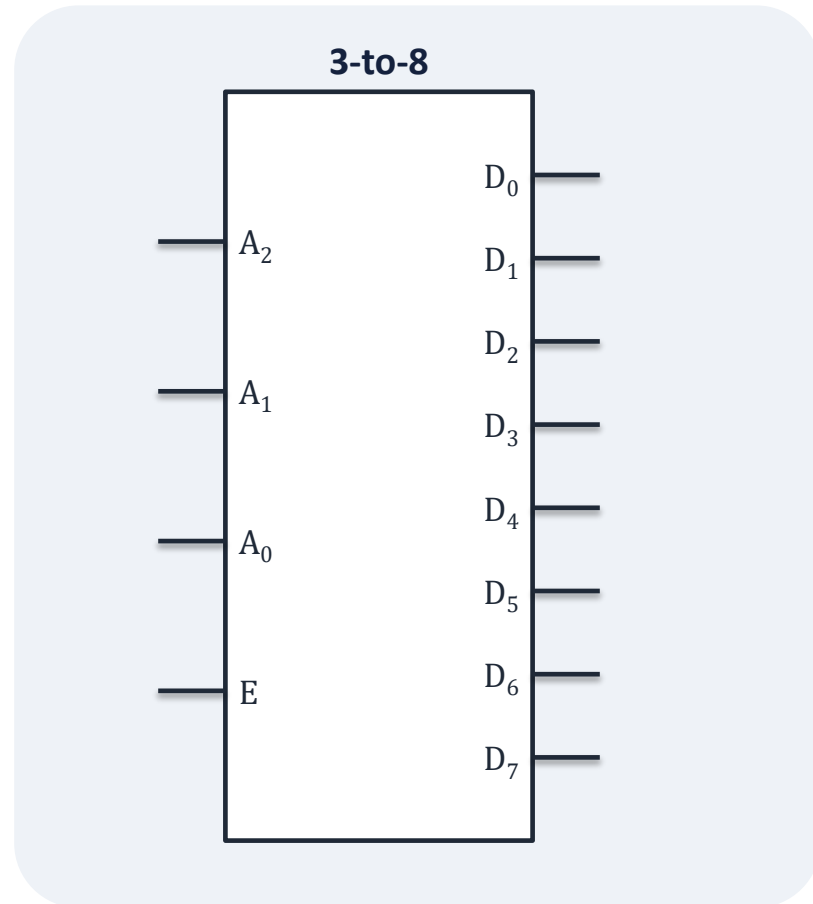
X means 'either value'. When disabled ($E' = 1$) every output is inactive (1). NAND gates build this version: it is how commercial decoders such as the 74LS139 work.

Example 1 — A 3-to-8 decoder

§4.9

EXAMPLE

Design a 3-to-8 decoder with an active-high enable E . Check the input 101.



Output	Minterm
D_0	$E A'_2 A'_1 A'_0$
D_1	$E A'_2 A'_1 A_0$
D_2	$E A'_2 A_1 A'_0$
D_3	$E A'_2 A_1 A_0$
D_4	$E A_2 A'_1 A'_0$
D_5	$E A_2 A'_1 A_0$
D_6	$E A_2 A_1 A'_0$
D_7	$E A_2 A_1 A_0$

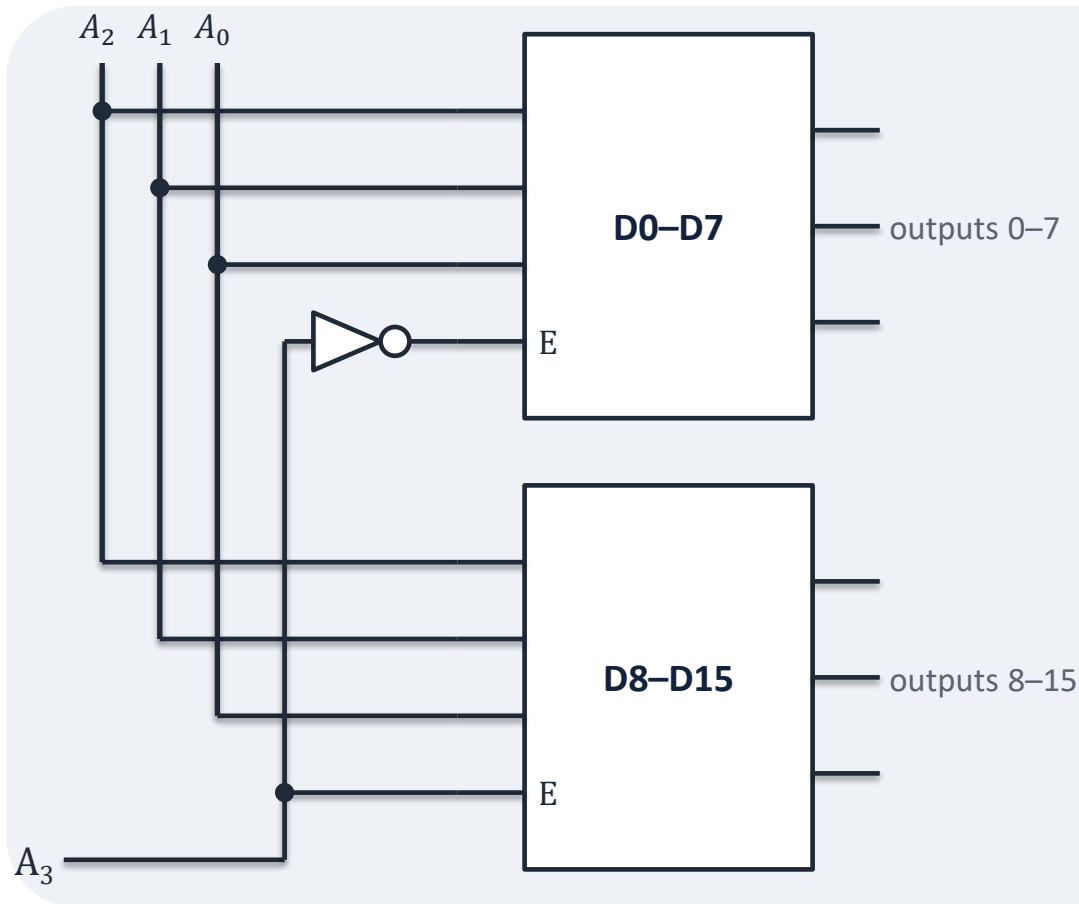
- 1 Each output: one minterm of $A_2 A_1 A_0$, ANDed with E
- 2 Build: 3 inverters and eight 4-input ANDs
- 3 Test 101 with $E = 1$: $A_2 = 1$, $A_1 = 0$, $A_0 = 1$
- 4 $D_5 = E A_2 A'_1 A_0 = 1 \cdot 1 \cdot 1 \cdot 1 = 1$
- 5 Every other AND has a 0 input, e.g. $D_1 = E A'_2 A'_1 A_0 = 0$
- 6 So only D_5 is 1 ✓
- 7 $E = 0$: every AND has a 0 input: all outputs 0

Example 2 — Decoder expansion

§4.9

EXAMPLE

Build a 4-to-16 decoder from two 3-to-8 decoders with enables. Which output does 1011 select?



- 1 16 outputs need 4 inputs; each decoder gives 8.
- 2 $A_2A_1A_0$ go to both decoders in parallel.
- 3 A_3 picks the half: A_3' enables the top, A_3 the bottom.
- 4 Input 1011: $A_3 = 1 \rightarrow$ top off, bottom on.
- 5 Bottom sees $A_2A_1A_0 = 011 = 3 \rightarrow$ its output 3.
- 6 Bottom outputs are 8-15: $8 + 3 \rightarrow D_{11} \checkmark$

Break

10 minutes — we resume at the 0:55 mark

Next: functions from a decoder, and encoders

Any function from a decoder

§4.9

The idea

A decoder produces every minterm of its inputs.

OR together the minterms of F : one OR gate per function.

Many 1s?

If F has more 1s than 0s, NOR the minterms of F' instead: fewer gate inputs.

Active-low outputs

Outputs are m'_j : use a NAND gate, since $(m'_1 m'_2)' = m_1 + m_2$.

When is it worth it?

One decoder serves every function of the same inputs.

Each extra function costs just one OR (or NAND) gate.

Best for several functions of few variables; a single simple function is cheaper as SOP.

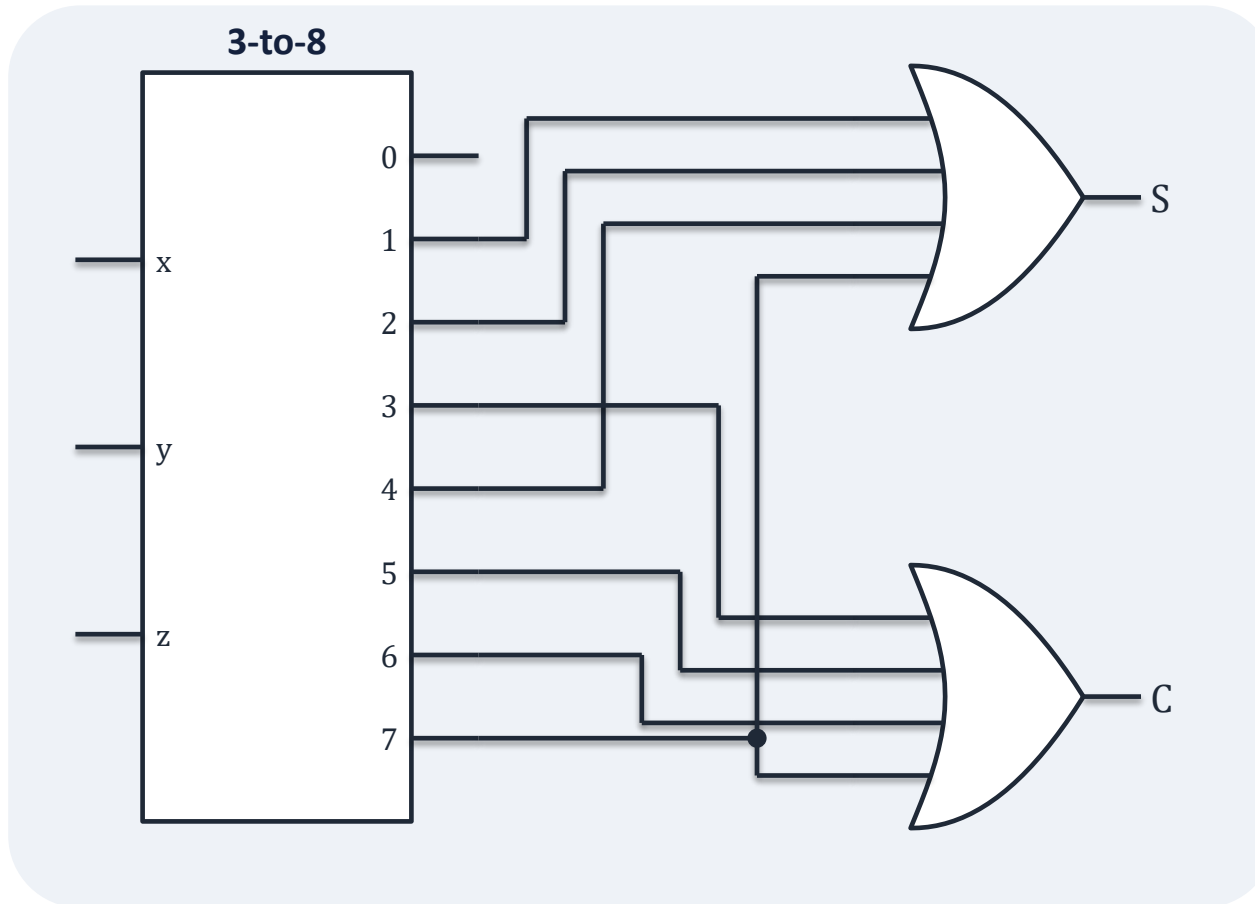
Every output can drive several gates, up to its fan-out.

Example 3 — Full adder from a decoder

§4.9

EXAMPLE

Implement a full adder with one 3-to-8 decoder and two OR gates. Check $xyz = 011$.



- 1 Full adder table: $S = \Sigma m(1,2,4,7)$, $C = \Sigma m(3,5,6,7)$
- 2 Connect x to the MSB input, then y , then z .
- 3 The decoder makes every minterm: $D_j = m_j$.
- 4 OR the minterms of S : $S = D_1 + D_2 + D_4 + D_7$
- 5 OR the minterms of C : $C = D_3 + D_5 + D_6 + D_7$
- 6 Check 011: only $D_3 = 1$, so $S = 0$, $C = 1$
- 7 $0 + 1 + 1 = 10_2$: sum 0, carry 1 ✓

Your turn 1 — Two functions

§4.9

YOUR TURN

≈ 10 min

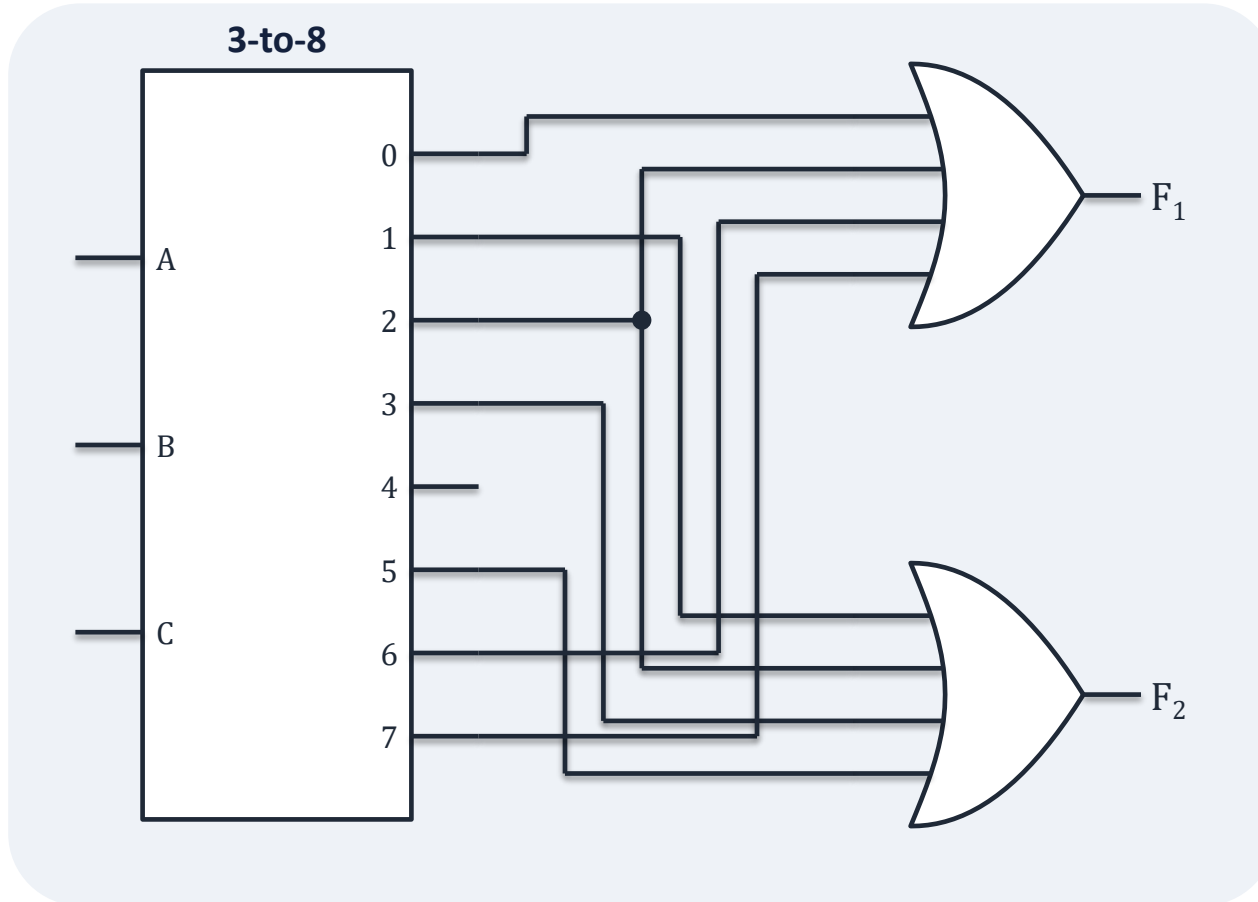
- (a) Implement $F_1(A, B, C) = \Sigma m(0, 2, 6, 7)$ and $F_2(A, B, C) = \Sigma m(1, 2, 3, 5)$ with one 3-to-8 decoder.
- (b) The decoder now has active-low outputs. What gate replaces each OR?
- (c) $F_3 = \Sigma m(0, 1, 2, 3, 4, 6)$ would need a 6-input OR. What cheaper gate does the job?

Count the gate inputs in (c).

Your turn 1 — Solution

§4.9

SOLUTION



- 1 (a) The decoder gives every minterm D_0 – D_7 of ABC .
- 2 $F_1 = \Sigma m(0,2,6,7)$: OR D_0, D_2, D_6, D_7
- 3 $F_2 = \Sigma m(1,2,3,5)$: OR D_1, D_2, D_3, D_5
- 4 D_2 feeds both OR gates.
- 5 (b) Outputs are D'_j . DeMorgan: $D_a + D_b = (D'_a D'_b)'$
- 6 so each OR becomes a **NAND**
- 7 (c) F_3 misses only 5 and 7: $F'_3 = m_5 + m_7$
- 8 $F_3 = (D_5 + D_7)'$: one 2-input NOR, not a 6-input OR

Encoders

§4.10

Active input	x	y	z
D0	0	0	0
D1	0	0	1
D2	0	1	0
D3	0	1	1
D4	1	0	0
D5	1	0	1
D6	1	1	0
D7	1	1	1

Octal-to-binary encoder

$$x = D_4 + D_5 + D_6 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7 \quad z = D_1 + D_3 + D_5 + D_7$$

Two problems

Two inputs active: D_3 and D_6 give 111, which is neither.

No input active gives 000, the same as D_0 .

Fix: a priority rule, plus a valid output V .

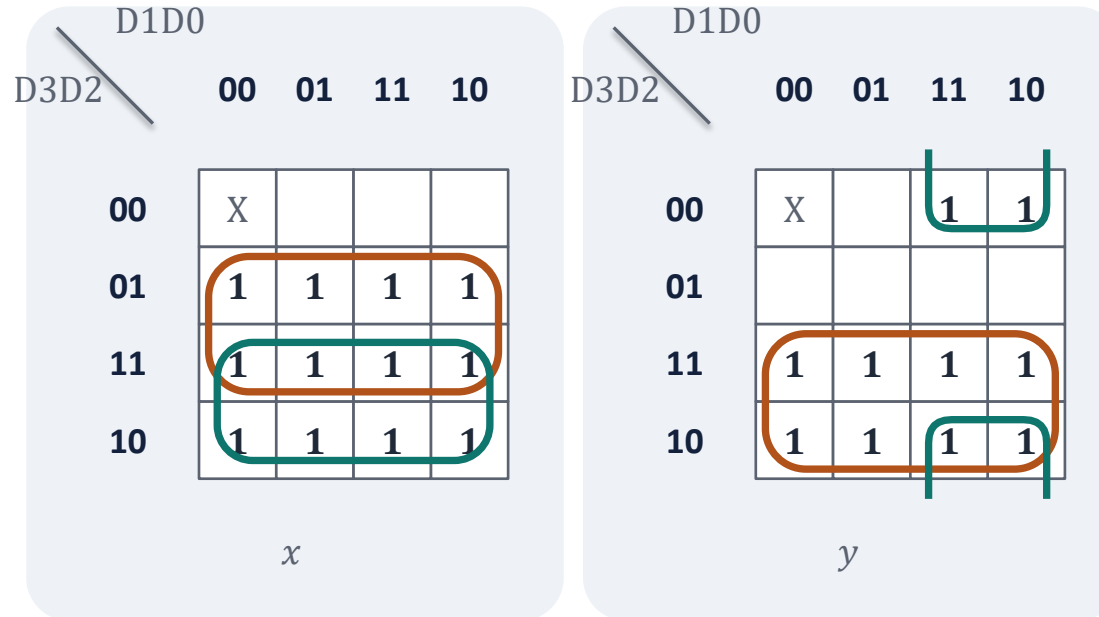
Example 4 — A priority encoder

§4.10

EXAMPLE

Design a 4-input priority encoder (D_3 highest priority) with outputs x , y and valid V .

D_3	D_2	D_1	D_0	x	y	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1



$$x = D_3 + D_2; \quad y = D_3 + D_2' D_1; \quad V = D_3 + D_2 + D_1 + D_0$$

Step by step on the next slide.

Example 4 — Step by step

§4.10

- 1 Fill the table from the top priority: $D_3 = 1 \rightarrow 11$ (rest X)
- 2 Else $D_2 = 1 \rightarrow 10$; else $D_1 = 1 \rightarrow 01$; else $D_0 = 1 \rightarrow 00$
- 3 All inputs 0: $V = 0$; x, y are don't-cares (m0)
- 4 x map: 1s wherever D_3 or D_2 is 1
- 5 Rows 01, 11 $\rightarrow D_2$; rows 11, 10 $\rightarrow D_3$
- 6 So $x = D_3 + D_2$
- 7 y map: 1s at m2, m3 (D_1 wins) and m8–m15 (D_3 wins)
- 8 Bottom half $\rightarrow D_3$; quad m2, m3, m10, m11 $\rightarrow D_2' D_1$
- 9 So $y = D_3 + D_2' D_1$; $V = D_3 + D_2 + D_1 + D_0$
- 10 Check 0110: $x = 0 + 1 = 1$, $y = 0 + 0 = 0 \rightarrow 10 = D_2$ ✓

Your turn 2 — Reverse priority

\$4.10

YOUR TURN

≈ 10 min

Redesign the 4-input priority encoder so that D_0 has the HIGHEST priority and D_3 the lowest.

- (a) Write the condensed truth table.
- (b) Find minimum expressions for x and y (use the all-zero row as a don't-care).
- (c) Does V change?

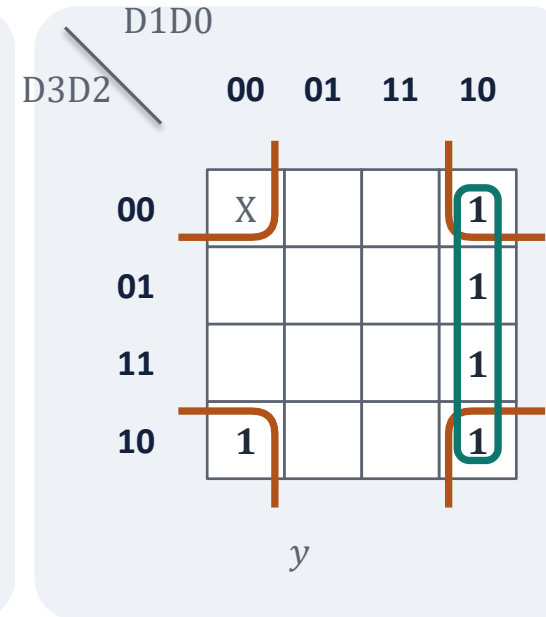
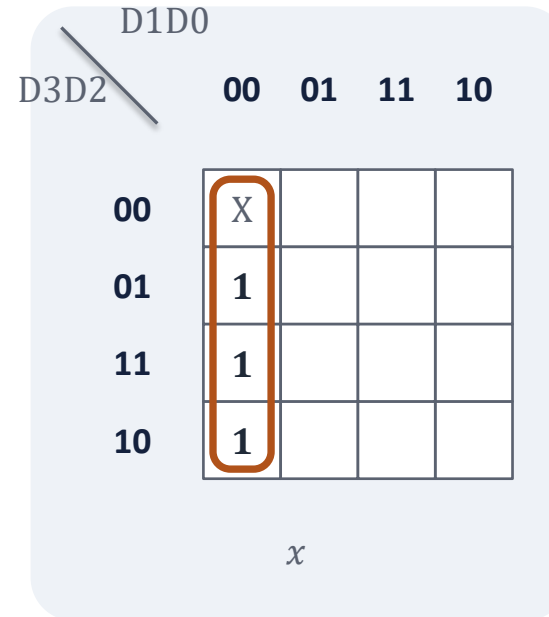
Map variables in the order $D_3D_2D_1D_0$.

Your turn 2 — Solution

§4.10

SOLUTION

D_3	D_2	D_1	D_0	x	y
0	0	0	0	X	X
X	X	X	1	0	0
X	X	1	0	0	1
X	1	0	0	1	0
1	0	0	0	1	1



$$x = D_1'D_0'$$

$$y = D_1D_0' + D_2'D_0'$$

$$V = D_3 + D_2 + D_1 + D_0$$

Steps on the next slide.

Your turn 2 — Step by step

§4.10

SOLUTION

- 1 (a) Top priority now D_0 : $D_0 = 1 \rightarrow 00$ (rest X)
- 2 Else $D_1 = 1 \rightarrow 01$; else $D_2 = 1 \rightarrow 10$; else $D_3 = 1 \rightarrow 11$
- 3 (b) x map: 1s at m4, m8, m12 and X at m0
- 4 Column $D_1D_0 = 00$ is a quad $\rightarrow x = D_1'D_0'$
- 5 y map: 1s at m2, m6, m10, m14 (D_1 wins) and m8
- 6 Column 10: $D_1 = 1, D_0 = 0 \rightarrow D_1D_0'$
- 7 m8: quad m0, m2, m8, m10 (X at m0) $\rightarrow D_2'D_0'$
- 8 So $y = D_1D_0' + D_2'D_0'$
- 9 (c) V is unchanged: 1 when any input is 1
- 10 Check 0110 (D_1 wins): $x = 0, y = 1 \rightarrow 01 \checkmark$

A decoder in Verilog

§4.12

Dataflow: 2-to-4, active-low enable

```
module dec2to4 (input  A, B, En,
                 output [3:0] D);
    assign D[0] = ~En & ~A & ~B;
    assign D[1] = ~En & ~A &  B;
    assign D[2] = ~En &  A & ~B;
    assign D[3] = ~En &  A &  B;
endmodule
```

En is active low; outputs active high. Each assign is one AND gate.

Behavioral alternative

```
assign D = En ? 4'b0000 : (4'b0001 << {A,B});
```

The shift puts a single 1 in position {A,B}.

Test it

Apply all 8 combinations of En, A, B and check that exactly one output is 1 whenever En = 0.

Concept check

1

Why is no minimization possible for a decoder's outputs?

2

Which gate combines active-low decoder outputs into a function?

3

What goes wrong in a plain encoder when two inputs are active?

4

What is the valid output V of a priority encoder for?

Key takeaways

Decoder

$$D_j = m_j$$

Active-low

$$D_j = m'_j \rightarrow \text{NAND}$$

Expansion

MSB drives the enables

Functions

decoder + one OR each

Encoder

$$x = D_4 + D_5 + D_6 + D_7$$

Priority

$$x = D_3 + D_2, y = D_3 + D_1 D'_2$$

Decoders select one of many; encoders name the one selected.

Homework #6 due at the start of Lecture 13

This afternoon's lab: Lab F — 2-to-4 decoder from NAND and AND gates

Next: multiplexers and three-state gates